X2-TURN: FRAME-SYNCHRONOUS DUAL-HEAD MODELING FOR JOINT STREAMING ASR AND TURN STATE PREDICTION

Kaiqi Fu, Rime Wen, Altman Lin, Shawn Qin, Roy Gan, Hao Wang, Qian Wang

X Square Robot
{kaiqifu, wanghao}@x2robot.com

arXiv:2608.10878v3 [cs.CL] 8 Sep 2026

ABSTRACT

Accurate and responsive turn-taking is essential for spoken dialogue systems, which must distinguish in real time between user interruptions, backchannels that should be ignored, and the completion of an utterance. Prior modular approaches typically optimize turn state prediction at the utterance or fixed-chunk level, creating a mismatch with the continuous turn state estimate, and often depend on an auxiliary ASR model, which limits responsiveness and increases overall system complexity. Therefore, we present X2-Turn [1], a frame-synchronous turn state prediction method via delayed-stream modeling. Specifically, building on the pretrained Voxtral Realtime model, we introduce a frame-synchronous turn state head that operates in parallel with the ASR head on shared streaming representations, jointly predicting ASR tokens and fine-grained turn states at the frame level. Experiments on bilingual EasyTurn and Full-Duplex-Bench demonstrate that the proposed method achieves an effective trade-off between turn state accuracy and decision latency.

***Index Terms*—** Turn-taking, spoken dialogue systems, delayed streams modeling, streaming automatic speech recognition

1. INTRODUCTION

Achieving natural spoken dialogue requires systems to seamlessly handle continuous speech, backchannels, and user interruptions while maintaining low latency [1]. To manage these complex conversational dynamics, a responsive system must continuously estimate fine-grained turn states. These states serve as the foundation for real-time dialogue control, determining when to interrupt text-to-speech (TTS) playback, take the conversational floor, or ignore a user backchannel.

Existing approaches broadly follow either end-to-end or cascaded paradigms. End-to-end full-duplex models [2–6] jointly learn speech understanding, interaction timing, and response generation. For example, Moshi employs a dual-stream architecture that jointly generates text and audio tokens for both speakers, using an inner-monologue mechanism to align semantic and acoustic generation [3]. Although these systems directly model synchronous interactions, jointly optimizing all components on limited full-duplex data may constrain the scale and general capabilities of the dialogue backbone.

Cascaded approaches instead decouple interaction control from response generation [7–12]. A representative example is the VAD–ASR–turn-detection pipeline [13], in which a front-end voice activity detection (VAD) module first segments the continuous user speech stream, an ASR model transcribes each resulting segment, and a semantic turn-detection model determines the turn state from the transcript. Because each stage depends on the output of the preceding stage, this pipeline introduces sequential latency and error propagation. To reduce the dependence on a separate upstream ASR module, subsequent studies have explored tighter integration between ASR and turn detection [8–11]. EasyTurn jointly predicts transcriptions and four turn states from VAD-segmented utterances, replacing the separate downstream turn-detection model with a unified model that reasons over ASR-derived transcripts [8]. JAL-Turn combines frozen SenseVoice and CPC representations to classify hold and shift states at candidate boundaries, whereas FastTurn integrates partial CTC hypotheses with acoustic cues to make low-latency decisions as the transcript is incrementally updated, without waiting for utterance completion [9, 10]. SoulX-Duplug further interleaves chunk-level ASR and state tokens within a single autoregressive stream, achieving strong turn-taking performance. However, it still relies on an external ASR model to guide state prediction during inference [11]. Despite these advances, these methods generally operate at the utterance or chunk level rather than continuously estimating the turn state at every frame. This mismatch in temporal granularity limits their responsiveness in real-time interactions.

More recently, Voxtral Realtime introduced a natively streaming ASR architecture based on delayed-stream modeling [3, 14]. It emits transcription tokens synchronously with the input audio at a fixed frame rate of 80 ms [15]. Inspired by this architecture, we introduce a turn state prediction head parallel to the ASR head, with both heads jointly optimized over shared causal decoder representations to predict ASR tokens and turn states simultaneously. To temporally align

[1] Code and models: https://github.com/X-Square-Robot/X2-Turn.

the two tasks, we propose ASR-anchored supervision, which projects word-level turn annotations onto the frame-level positions of the corresponding ASR tokens. Experiments on the Chinese and English EasyTurn test sets demonstrate that the proposed method achieves an effective trade-off between turn state accuracy and decision latency.

Our main contributions are summarized as follows:

1. We propose X2-Turn, which extends a pretrained delayed-stream ASR model with a parallel turn state head, enabling joint frame-synchronous ASR and turn state prediction within a single streaming forward pass.

2. We design a unified turn state label set that supports interruption, turn completion, and backchannel detection. We further introduce an ASR-anchored supervision method that projects word-level turn annotations onto the frame-level ASR token timeline.

3. We conduct bilingual experiments on EasyTurn and Full-Duplex-Bench [16, 17], validating the effectiveness of the proposed method for streaming turn state prediction under different latency settings controlled by τ.

2. METHOD

In this section, we first introduce the streaming ASR backbone and dual-head architecture, and then describe the turn state token design and the construction of ASR-anchored turn state labels. An overview of the framework is shown in Fig. 1.

2.1. Dual-Head Modeling

Our goal is to estimate turn states synchronously with ASR transcription as speech unfolds, while keeping interaction control independent of the downstream dialogue model. To this end, we build our system on Voxtral Realtime [15], a natively streaming ASR model based on delayed-stream modeling (DSM). Voxtral Realtime maps an input waveform to a delayed token stream using a causal audio encoder and a language decoder. We retain the backbone architecture and introduce a parallel turn state prediction head. This design decouples turn state estimation from response generation, allowing the resulting turn-taking module to be integrated into different dialogue systems.

The architecture of the proposed method is illustrated in Fig. 1. We first briefly review the Voxtral Realtime backbone before presenting our dual-head extension. Voxtral Realtime consists of three main components: a causal audio encoder that maps 16-kHz waveforms to frame-level audio features, a temporal adapter that downsamples the encoder features to 12.5 Hz, and a decoder-only language model that emits one token at each 80-ms step. At step i, the decoder takes as input the sum of the current audio embedding and the text embedding of the previously emitted ASR token y_{i-1}^{asr}. It then produces a hidden state h_i, from which the ASR head predicts the next token. The ASR head is trained using token-level cross-entropy over the ASR vocabulary:

$$\mathcal{L}_{\text{asr}} = -\sum_{i=1}^{T} \log P_{\text{asr}} \left(y_i^{\text{asr}} \mid x_{\le i}, y_{<i}^{\text{asr}} \right), \tag{1}$$

where $x_{\le i}$ denotes the audio observed up to step i, $y_{<i}^{\text{asr}}$ denotes the previously emitted ASR tokens, and T is the sequence length.

In addition to ordinary subword tokens, the ASR vocabulary contains two special symbols: a padding token `[P]` and a word-boundary token `[W]`. The output token stream is delayed relative to the input audio by a configurable target delay τ. This delay is conditioned into the decoder through AdaRMSNorm, allowing a single model to operate at delays corresponding to different multiples of 80 ms.

We preserve the original ASR prediction head and add a parallel turn state prediction head. Both heads operate on the shared hidden state h_i, enabling ASR and turn state prediction within a single forward pass. The two heads are jointly optimized using the following objective:

$$\mathcal{L}_{\text{turn}} = -\sum_{i=1}^{T} \log P_{\text{turn}} \left(y_i^{\text{turn}} \mid x_{\le i}, y_{<i}^{\text{asr}} \right), \tag{2}$$

$$\mathcal{L} = \mathcal{L}_{\text{asr}} + \lambda \mathcal{L}_{\text{turn}}, \tag{3}$$

where y_i^{turn} denotes the ground-truth turn state at step i, and λ controls the contribution of the turn state loss. For notational simplicity, the target delay τ is omitted from the conditional distributions above ($\tau = 0$).

At inference time, autoregressive decoding is driven solely by the ASR head. At each step, the turn head independently predicts the turn state from the shared hidden state h_i, and its prediction is not fed back into the decoding loop. Therefore, turn state prediction errors do not affect subsequent ASR decoding. Both transcription and turn states are produced within the same forward pass, without introducing an additional sequential inference stage and latency.

2.2. Turn-Taking State Token Design

We define five turn state tokens to represent the evolving state of a user turn:

- `<|idle|>` represents user silence, corresponding to non-speech segments derived from forced alignment.
- `<|noidle|>` indicates active speech without semantic content (e.g., the initial syllables of an utterance).
- `<|incomplete|>` denotes active speech containing partial semantic content.

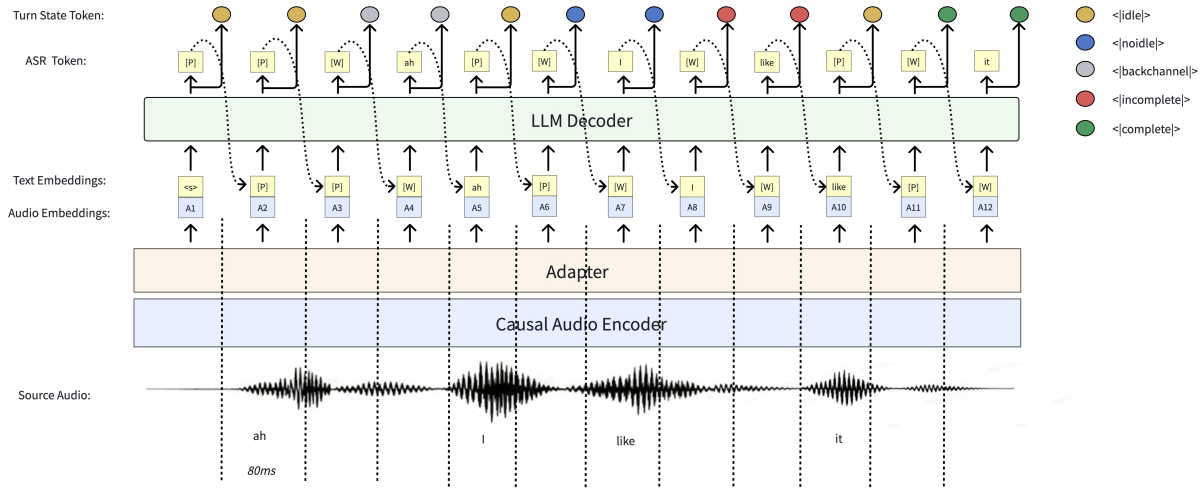


Fig. 1. Overview of X2-Turn, the proposed frame-synchronous dual-head architecture, with a target delay of $\tau = 80\,\mathrm{ms}$. Once the onset of a word has been observed and the target delay has elapsed, the ASR head emits a word-boundary token `[W]`, while the turn state head simultaneously predicts the corresponding state. Subsequent subword tokens are then emitted frame by frame.

- `<|complete|>` signifies active speech with complete semantic content.
- `<|backchannel|>` captures user backchannel signals or filler words (e.g., "um", "ah").

Unlike the definition adopted in prior work [11], our `<|noidle|>` state refers specifically to early active speech for which sufficient semantic content has not yet been observed. It does not represent non-speech or background noise. This distinction allows the model to separate true silence from the early portion of an ongoing user utterance.

2.3. ASR-Anchored Turn State Supervision

We next describe the construction of paired ASR and turn state targets. We first construct the delayed-stream ASR target sequence. Let the i-th word span the time interval $[s_i, e_i]$. Each word is tokenized into one or more subword tokens, for example, multiple BPE tokens for an English word. Regardless of the number of subword tokens, each word is represented by a single word-boundary token `[W]` followed by its corresponding subword tokens.

Given the onset time s_i, the target position p_i of the word-boundary token is computed as

$$p_i = \mathrm{round}\left(\frac{s_i}{\Delta}\right) + n_\tau, \tag{4}$$

where $\Delta = 80\,\mathrm{ms}$ is the frame duration and $n_\tau = \tau/\Delta$ is the target delay measured in frames.

Unlike the original Voxtral formulation, which places `[W]` according to the word offset, we anchor `[W]` to the word onset. The word's subword tokens are placed immediately after `[W]`, while all remaining positions are filled with the padding token `[P]`. This onset-based placement makes the ASR and turn state predictions available earlier relative to the spoken word. Because each word occupies one `[W]` position followed by at least one subword position, its representation requires at least two 80-ms steps.

Turn state targets are constructed using the same placement procedure. We first prompt a powerful language model [18] to assign a turn state label to each word. The resulting word-level label is then assigned to all target positions occupied by the corresponding `[W]` token and subword tokens. All unoccupied positions are labeled as `<|idle|>`, including positions corresponding to pre-speech silence, inter-word pauses, mid-utterance pauses, and trailing silence. In this way, the ASR and turn state targets are aligned on the same 80-ms discrete timeline.

This ASR-anchored supervision scheme has three main properties. First, each word-level turn state label is placed at the same positions as its corresponding ASR tokens. The turn head therefore predicts the state from the same decoder representations used by the ASR head to emit the associated transcription. Second, silence is explicitly supervised using the `<|idle|>` state, allowing a downstream interaction policy to perform endpointing by counting consecutive idle frames following a `<|complete|>` prediction. Third, because the ASR and turn state targets are aligned position by position rather than at the utterance or chunk level, both tasks are optimized over the same frame-synchronous discrete timeline.

Table 1. System-level results on bilingual Full-Duplex-Bench. ↑/↓ indicate higher-/lower-better. Best per language in **bold**; second-best underlined.

Model	Pause	Turn-Taking		Backchannel	Interruption v1		Interruption v1.5			Overall	
	TOR↓	TOR↑	RL↓	RsR↑	TOR↑	RL↓	RpR↑	SL↓	RL↓	ACC↑	Latency↓
English											
dGSLM	0.935	0.975	0.352	–	0.917	2.531	–	–	–	0.653	1.442
PersonaPlex	0.623	**0.992**	**0.070**	–	**1.000**	0.400	–	–	–	0.790	**0.235**
Moshi	0.983	0.941	0.265	0.060	**1.000**	**0.257**	0.500	1.160	1.470	0.504	0.788
Freeze-Omni	0.562	0.336	0.953	0.800	0.867	1.409	0.720	1.420	1.350	0.632	1.283
Gemini Live	0.283	0.655	1.301	**0.930**	0.891	1.183	0.330	2.200	2.620	0.705	1.826
GPT-4o	–	–	–	0.700	–	–	0.780	**0.230**	1.500	0.740	0.865
SoulX-Duplug	0.352	0.933	0.511	0.740	0.970	0.773	0.770	0.450	1.030	0.812	0.691
X2-Turn (Ours)	**0.224**	0.807	0.901	0.925	0.935	0.659	**0.855**	0.574	**0.942**	**0.860**	0.769
Chinese											
Freeze-Omni	0.042	0.652	0.780	0.700	0.975	**0.232**	0.440	1.300	1.630	0.745	0.986
SoulX-Duplug	0.038	**0.994**	**0.767**	0.800	0.994	1.089	0.830	**0.380**	1.150	0.916	**0.847**
X2-Turn (Ours)	**0.033**	**0.994**	0.888	**0.980**	**1.000**	1.165	**0.932**	0.578	**0.979**	**0.975**	0.903

3. EXPERIMENTAL SETUP

3.1. Data Preparation

The corpora used in this work consist of two parts: Chinese-English ASR data and turn-taking data. For the ASR data, we use AISHELL 1∼4 [19–22], AliMeeting [23], WenetSpeech [24], KeSpeech [25], LibriSpeech [26], GigaSpeech [27], TED-LIUM [28], and VoxPopuli [29], totaling approximately 26k hours (14k hours in Chinese and 12k hours in English). This portion of data is used in Stage 1 to strengthen the model's ASR capability in both languages. For the turn-taking data, we select a subset of the EasyTurn training set for Chinese (approximately 126 hours) and a subset of Fisher [30] telephone conversations for English (approximately 249 hours). This portion of data is used in Stage 2 for joint ASR and turn-taking modeling.

For both parts of the data, Qwen3-ForceAligner [31] was employed to obtain word-level timestamps. In addition, for the turn-taking data, we use Qwen3.5-Plus as an LLM annotator to perform word-level semantic turn state labeling, following the annotation criteria defined in Section 2.3. Word-level labels are then projected onto the 80 ms frame delayed-stream positions of their ASR word markers `[W]` and subword tokens.

3.2. Implementation Details

We use Voxtral-Mini-4B-Realtime [2] as the pretrained streaming backbone. All experiments fully fine-tune both the causal audio encoder and the language decoder. Training proceeds in two stages, with the ASR streaming delay τ sampled per batch between 1 and 30 frames (80–2400 ms) in both stages, so that a single model covers a range of latency configurations.

During Stage 1, streaming ASR adaptation adapts the backbone to the delayed-stream ASR protocol on a large-scale Chinese–English corpus with frame-level ASR labels. Stage 2 then performs joint ASR and turn state fine-tuning. A turn state head is added, initialized as a copy of the ASR head, and the full model is fine-tuned on the Chinese–English turn-taking training set with paired ASR and frame-level turn state labels. The joint objective uses $\lambda = 0.1$.

3.3. Latency Metric

We measure *latency* relative to the end of each word. For a word spanning $[s_i, e_i]$, the corresponding turn state becomes available at time $s_i + \tau$, i.e., after the configured streaming delay τ from the word onset. The resulting latency relative to the end of the word is

$$L_i \;=\; \tau - (e_i - s_i), \tag{5}$$

which measures how long after the user finishes speaking word i the corresponding turn decision becomes available. If the word duration exceeds τ, L_i can be negative, meaning the state is available before the word ends. We report the average L_i over all words in the EasyTurn test set. For cascaded baselines, following prior work [11], we report inference time plus the front-end VAD delay $\mathrm{latency}_{\mathrm{vad}}$. While not strictly identical to L_i, this reflects their end-to-end decision delay.

[2] https://huggingface.co/mistralai/Voxtral-Mini-4B-Realtime-2602

Table 2. Module-level results on EasyTurn. $\mathrm{ACC}_{\mathrm{comp}}$, $\mathrm{ACC}_{\mathrm{incomp}}$, and $\mathrm{ACC}_{\mathrm{bc}}$ denote utterance-level accuracy for the complete, incomplete, and backchannel categories, respectively ($\mathrm{ACC}_{\mathrm{bc}}$ reported for Chinese only, as English has no backchannel split). "–" denotes an unsupported or unavailable state, and $\mathrm{latency}_{\mathrm{vad}}$ denotes the front-end VAD delay incurred by cascaded methods.

Method	**Streaming**	**ACC**$_{\mathrm{comp}}$ (%, ↑)	**ACC**$_{\mathrm{incomp}}$ (%, ↑)	**ACC**$_{\mathrm{bc}}$ (%, ↑)	**Latency** (ms, ↓)
Chinese					
Paraformer + TEN Turn	✗	86.67	89.30	–	$\mathrm{latency}_{\mathrm{vad}}$ + 204
Smart Turn V3	✗	91.33	60.00	–	$\mathrm{latency}_{\mathrm{vad}}$ + 24
EasyTurn	✗	96.33	97.67	91.00	$\mathrm{latency}_{\mathrm{vad}}$ + 263
SoulX-Duplug	✓	89.33	79.33	–	295
X2-Turn (Ours)	✓	**91.00**	**93.00**	**96.00**	288
English					
SenseVoice En + TEN Turn	✗	95.60	76.59	–	$\mathrm{latency}_{\mathrm{vad}}$ + 57
Smart Turn V3	✗	78.93	72.24	–	$\mathrm{latency}_{\mathrm{vad}}$ + 21
SoulX-Duplug	✓	77.67	88.96	–	205
X2-Turn (Ours)	✓	**92.10**	**84.60**	–	225

4. RESULTS AND ANALYSIS

We evaluate the proposed method at two levels. Section 4.1.1 reports system-level full-duplex behavior on bilingual Full-Duplex-Bench. Section 4.1.2 then isolates the streaming turn module on EasyTurn. Sections 4.2 and 4.3 ablate word-boundary placement and the streaming delay τ. Section 4.4 reports streaming ASR quality.

4.1. Main Results

4.1.1. System results on bilingual Full-Duplex-Bench

We build a cascaded full-duplex dialogue system by coupling X2-Turn, which handles speech transcription and dialogue state management, with Qwen2.5-7B-Instruct [32] for response generation and Qwen3-TTS-Streaming [3] for speech synthesis. We evaluate the complete system on Bilingual Full-Duplex-Bench [16, 17], with results summarized in Table 1. The gains are distributed across tasks rather than concentrated in one. Pause TOR is the lowest among all systems in both languages, indicating that X2-Turn takes the floor least often during intra-turn pauses. Backchannel RsR is highest in Chinese and second only to Gemini Live in English. Interruption RpR is highest in both languages, while interruption v1 TOR is competitive but not top-ranked. Chinese turn-taking TOR matches SoulX-Duplug.

The remaining gap lies in English turn-taking TOR and a slightly higher overall latency relative to SoulX-Duplug. SoulX-Duplug takes the floor as soon as the model emits `<|complete|>`, applying no additional decision rule, which yields very low response latency. We instead require a short run of `<|idle|>` frames following `<|complete|>` before taking the turn. This conservative endpointing rule reduces false take-overs during pauses, at the cost of lower English turn-taking TOR and a small added delay.

[3] https://github.com/X-Square-Robot/Qwen3TTS-Streaming

Latency remains competitive overall. X2-Turn avoids both the extreme pause TOR of Moshi and the multi-second interruption delays of Gemini Live. Among systems with fully reported metrics, X2-Turn therefore achieves the strongest overall ACC, with latency close to that of SoulX-Duplug.

4.1.2. Module results on bilingual EasyTurn

Table 2 compares X2-Turn (with $\tau = 480\,ms$) against baselines on EasyTurn in terms of turn state classification accuracy and latency. We compare the last non-idle predicted state against the ground-truth utterance.

Among fully streaming systems, X2-Turn outperforms SoulX-Duplug on both accuracy metrics in Chinese, with similar latency (slightly lower). In English, X2-Turn achieves substantially higher complete accuracy (92.10 vs. 77.67) but lower incomplete accuracy (84.60 vs. 88.96), at a latency cost of 20ms. It also requires no auxiliary ASR model at inference: a single model jointly decodes transcripts and turn states.

Cascaded, VAD-dependent systems can post higher complete and incomplete accuracy, especially EasyTurn on Chinese, because they classify a full VAD segment. That segmentation adds a front-end delay that is not included in the raw inference times [7], so the higher accuracy does not imply a streaming responsiveness advantage. X2-Turn remains fully streaming and improves Chinese backchannel accuracy over EasyTurn (96.00 vs. 91.00).

4.2. Onset vs. Offset Word-boundary Placement

Voxtral places `[W]` at the word offset; we instead anchor it at the onset (Section 2). Because the turn state is aligned with `[W]`, this choice also sets when the decision can first appear. A delayed-stream model waits τ after that anchor. Offset therefore cannot emit until the word has been fully heard

Table 3. Onset vs. offset [W] placement on the EasyTurn testset. Offset uses a smaller τ to match latency after the word ends. Since we do not define an explicit <|wait|> label, $\mathrm{ACC}_{\mathrm{wait}}$ is computed using semantic completeness together with the <|complete|> state.

Placement	τ (ms)	$\mathrm{ACC}_{\mathrm{comp}}$	$\mathrm{ACC}_{\mathrm{incomp}}$	$\mathrm{ACC}_{\mathrm{bc}}$	$\mathrm{ACC}_{\mathrm{wait}}$	Avg.	Lat. (ms)
Chinese							
Offset	240	90.33	91.67	93.00	96.00	91.88	**240**
Offset	320	**93.33**	88.67	92.00	96.00	91.75	320
Onset	480	91.00	**93.00**	**96.00**	**98.00**	**93.25**	288
English							
Offset	240	87.74	83.61	–	–	85.74	240
Offset	320	86.48	80.27	–	–	83.47	320
Onset	480	**92.10**	**84.60**	–	–	**88.49**	**225**

plus τ, whereas onset can decide while the speaker is still finishing the word. The two cannot be compared at the same τ. Table 3 matches them on latency after the word ends (Eq. 5), using a larger τ for onset.

Onset is stronger overall in both languages and earlier in Chinese. Offset, having heard the full word, slightly improves Chinese complete accuracy; onset is better on incomplete, backchannel, and wait, i.e., less likely to take the floor too soon. On English, offset underperforms onset at both delays, and extra delay after the word end does not close the gap. The modest drop from 240 ms to 320 ms may also reflect training variance. We therefore keep onset in the X2-Turn system.

Table 4. Ablation on the delay τ on the EasyTurn testsets.

	τ (ms)	$\mathrm{ACC}_{\mathrm{comp}}$	$\mathrm{ACC}_{\mathrm{incomp}}$	Avg.	Latency (ms)
Chinese	480	91.00	93.00	92.00	288
	400	88.70	94.30	91.50	208
	320	87.33	94.00	90.67	128
English	480	92.10	84.60	88.49	225
	400	85.20	85.30	85.25	145
	320	82.70	87.60	85.09	65

4.3. Effect of Streaming Delay τ for Turn-taking

We report the effect of different values of the streaming delay τ on both turn-taking and ASR performance. We first examine the effect of τ on turn-taking performance, considering three configurations (320, 400 and 480 ms), as turn-taking inherently favors low-latency settings.

Table 4 shows a consistent latency–accuracy trade-off across both languages. As τ decreases from 480 ms to 320 ms, latency drops substantially, while turn state accuracy degrades only mildly. This indicates that our model is robust under tight latency constraints, allowing a suitable operating point to be chosen for different latency requirements with little loss in turn-taking accuracy.

Table 5. Streaming ASR word error rate (WER, %, ↓). Uni-ASR uses a 320 ms chunk with beam search; Freeze-Omni uses a chunk size of 4. "–" denotes unreported results.

Test set	Uni-ASR [33]	Freeze-Omni [4]	Stage1		Stage2
			480 ms	2400 ms	480 ms
Chinese					
AISHELL-1	2.90	2.79	2.57	1.48	3.94
AISHELL-2 Android	–	–	4.70	3.42	5.70
AISHELL-2 iOS	–	–	4.41	3.23	5.36
AISHELL-2 Mic	–	–	4.54	3.54	5.55
AISHELL-3	–	–	3.16	2.13	4.66
AISHELL-4	–	–	18.72	16.42	22.79
WenetSpeech Meeting	–	14.2	9.25	7.68	12.18
WenetSpeech Net	–	12.6	9.50	8.40	10.39
English					
GigaSpeech	–	–	12.23	10.87	12.55
LibriSpeech Clean	3.21	4.05	2.40	1.54	3.30
LibriSpeech Other	7.71	10.48	5.87	3.77	8.53
TED-LIUM	–	–	4.62	3.66	4.50
VoxPopuli	–	–	9.16	6.05	11.91

4.4. Streaming ASR Performance Comparison

In this subsection, we compare our frame-synchronous approach against representative chunk-based streaming ASR systems. Since SoulX-Duplug does not report ASR results, we instead select two strong chunk-based streaming baselines, Uni-ASR and Freeze-Omni, for comparison. Table 5 summarizes the results. Under comparable or lower delay settings, Stage1-ASR at τ=480 ms already outperforms both baselines across nearly all evaluation sets. This indicates that frame-wise prediction with a short lookahead is competitive with, and often superior to fixed-chunk streaming even before accounting for its lower latency. Even after Stage2-Turn joint training, where turn-state supervision introduces a set-dependent degradation relative to Stage1-ASR at the same delay, our model still matches or exceeds Freeze-Omni. This suggests that the frame-synchronous backbone retains most of its recognition advantage over chunk-based streaming even under multi-task optimization.

5. CONCLUSION

This paper presents X2-Turn, a frame-synchronous dual-head extension of a pretrained delayed-stream ASR model for joint streaming ASR and turn state prediction. A parallel turn state head shares causal decoder representations with the ASR head, with ASR-anchored supervision projecting word-level turn labels onto the native 80 ms token timeline. The streaming delay τ offers a controllable trade-off between turn-taking accuracy, response latency, and ASR quality. Experiments on bilingual EasyTurn and Full-Duplex-Bench demonstrate that X2-Turn achieves accurate turn-taking detection while maintaining low latency. Future work will further balance the ASR and turn state objectives and improve robustness in more challenging conversational settings.

6. REFERENCES

[1] G. Skantze, "Turn-taking in conversational systems and human-robot interaction: a review," *Computer Speech & Language*, vol. 67, pp. 101178, 2021.

[2] T. A. Nguyen et al., "Generative spoken dialogue language modeling," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 250–266, 2023.

[3] A. Défossez et al., "Moshi: a speech-text foundation model for real-time dialogue," *arXiv preprint arXiv:2410.00037*, 2024.

[4] X. Wang et al., "Freeze-omni: A smart and low latency speech-to-speech dialogue model with frozen LLM," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.

[5] Q. Zhang et al., "Omniflatten: An end-to-end gpt model for seamless voice conversation," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 14570–14580.

[6] R. Roy et al., "Personaplex: Voice and role control for full duplex conversational speech models," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2026, pp. 16137–16141.

[7] B. Liao et al., "Flexduo: A pluggable system for enabling full-duplex capabilities in speech dialogue systems," *arXiv preprint arXiv:2502.13472*, 2025.

[8] G. Li et al., "Easy turn: Integrating acoustic and linguistic modalities for robust turn-taking in full-duplex spoken dialogue systems," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2026, pp. 16957–16961.

[9] G. Yang, Y. Pan, S. Qiu, and N. Bai, "Jal-turn: Joint acoustic-linguistic modeling for real-time and robust turn-taking detection in full-duplex spoken dialogue systems," *arXiv preprint arXiv:2603.26515*, 2026.

[10] C. Wang et al., "Fastturn: Unifying acoustic and streaming semantic cues for low-latency and robust turn detection," *arXiv preprint arXiv:2604.01897*, 2026.

[11] R. Yan et al., "Soulx-duplug: Plug-and-play streaming state prediction module for realtime full-duplex speech conversation," *arXiv preprint arXiv:2603.14877*, 2026.

[12] JD.com, "JoyAI-Talker: Full-duplex speech interactive large model built for empathetic voice agents," *arXiv preprint arXiv:2608.01119*, 2026.

[13] T. Team, "Ten vad: A low-latency, lightweight and high-performance streaming voice activity detector (vad)," GitHub repository: https://github.com/TEN-framework/ten-vad, 2025.

[14] N. Zeghidour et al., "Streaming sequence-to-sequence learning with delayed streams modeling," *arXiv preprint arXiv:2509.08753*, 2025.

[15] A. H. Liu et al., "Voxtral realtime," *arXiv preprint arXiv:2602.11298*, 2026.

[16] G.-T. Lin et al., "Full-duplex-bench: A benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities," in *2025 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2025, pp. 1–8.

[17] G.-T. Lin et al., "Full-duplex-bench v1. 5: Evaluating overlap handling for full-duplex speech models," in *ICASSP 2026-2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2026.

[18] A. Yang et al., "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[19] H. Bu, J. Du, X. Na, B. Wu, and H. Zheng, "Aishell-1: An open-source mandarin speech corpus and a speech recognition baseline," in *Proc. O-COCOSDA*, 2017, pp. 1–5.

[20] J. Du, X. Na, X. Liu, and H. Bu, "Aishell-2: Transforming mandarin asr research into industrial scale," *arXiv preprint arXiv:1808.10583*, 2018.

[21] Y. Shi, H. Bu, X. Xu, S. Zhang, and M. Li, "Aishell-3: A multi-speaker mandarin tts corpus and the baselines," *arXiv preprint arXiv:2010.11567*, 2020.

[22] Y. Fu et al., "Aishell-4: An open source dataset for speech enhancement, separation, recognition and speaker diarization in conference scenario," *arXiv preprint arXiv:2104.03603*, 2021.

[23] F. Yu et al., "M2met: The icassp 2022 multi-channel multi-party meeting transcription challenge," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2022, pp. 6167–6171.

[24] B. Zhang et al., "Wenetspeech: A 10000+ hours multi-domain mandarin corpus for speech recognition," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2022, pp. 6182–6186.

[25] Z. Tang et al., "Kespeech: An open source speech dataset of mandarin and its eight subdialects," in *Proc. NeurIPS Datasets and Benchmarks*, 2021.

[26] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: an asr corpus based on public domain audio books," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2015, pp. 5206–5210.

[27] G. Chen et al., "Gigaspeech: An evolving, multi-domain asr corpus with 10,000 hours of transcribed audio," *arXiv preprint arXiv:2106.06909*, 2021.

[28] F. Hernandez, V. Nguyen, S. Ghannay, N. Tomashenko, and Y. Esteve, "Ted-lium 3: Twice as much data and corpus repartition for experiments on speaker adaptation," in *Proc. Int. Conf. Speech Comput. (SPECOM)*, 2018, pp. 198–208.

[29] C. Wang et al., "Voxpopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation," in *Proc. ACL-IJCNLP*, 2021, pp. 993–1003.

[30] C. Cieri, D. Miller, and K. Walker, "The fisher corpus: A resource for the next generations of speech-to-text.," in *LREC*, 2004, vol. 4, pp. 69–71.

[31] X. Shi et al., "Qwen3-asr technical report," *arXiv preprint arXiv:2601.21337*, 2026.

[32] Q. Team, "Qwen2.5: A party of foundation models," September 2024.

[33] Y. Xia, J. Tang, J. Hou, G. Xu, and H. Yao, "Uni-asr: Unified llm-based architecture for non-streaming and streaming automatic speech recognition," *arXiv preprint arXiv:2603.11123*, 2026.